

Entregable 1: semáforos en Lustre

ACTIVIDAD 1: Realizar el código del ejercicio.

Para abordar la solución de este primer apartado, hemos considerado fundamental realizar una fase previa de análisis y diseño antes de proceder directamente con la programación. Dado que Lustre es un lenguaje síncrono basado en el flujo de datos, resulta imprescindible comprender detalladamente cómo va a evolucionar el sistema paso a paso.

Por ello, el primer paso ha consistido en trazar un esquema temporal del funcionamiento y la sincronización que deberán mantener ambos semáforos (S1 para la vía principal y S2 para la vía secundaria). En concreto, nos hemos centrado en modelar la secuencia completa que se desencadena en el momento en que el detector de presencia de vehículos (carSensor) pasa a estado activo. El resultado de este diseño preliminar se plasma en el siguiente esquema:

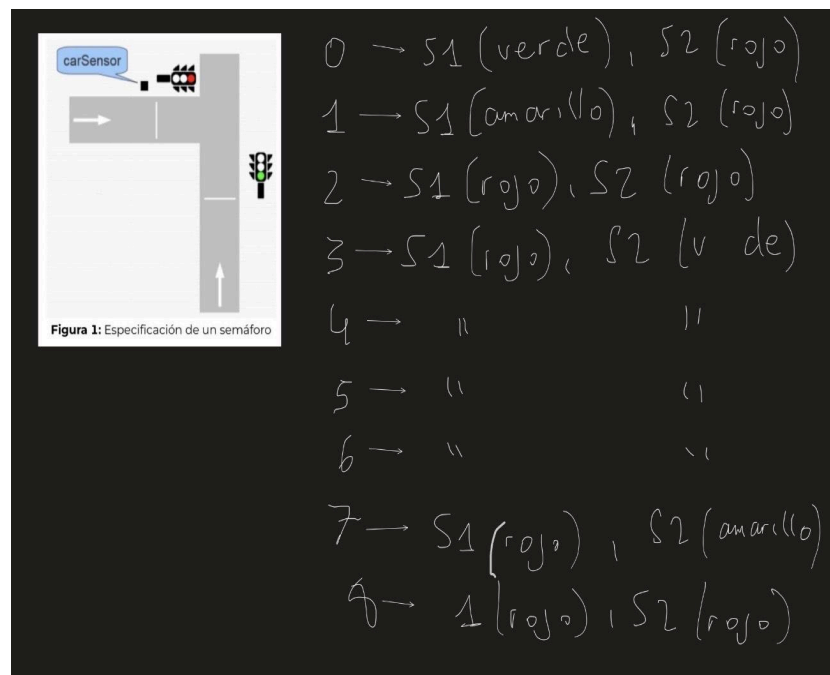


Imagen 1. Diseño funcionamiento

NOTA: Los números a la izquierda de las flechas es el tick de reloj en el que nos encontramos. A la derecha de las flechas lo que tenemos son los semáforos y su estado entre paréntesis en cada tick.

Tomando como referencia directa este modelo temporal estructurado, la traducción matemática al lenguaje Lustre resulta mucho más factible.

A continuación se detallan los posibles escenarios que se pueden dar y cómo se implementan cada uno en el código:

```

preFase = 0 -> pre(Fase);
Fase = if carSensor and (preFase = 0) then 1
      else if 0 < preFase and preFase < 8 then preFase + 1
      else 0;

verdeS1 = Fase = 0;
verdeS2 = Fase > 2 and Fase < 7;

amarilloS1 = Fase = 1;
amarilloS2 = Fase = 7;

rojoS1 = Fase > 1;
rojoS2 = (Fase < 3) or (Fase = 8);

```

Imagen 2. Parte código actividad 1

- 1) Estado inicial: Mientras el carSensor no se active, los semáforos van a mantener los estados de S1(verde), S2(rojo).

- Esto lo pudimos implementar a través del “else 0” de la línea 4 de la foto, que dice que si el carSensor no se activa la Fase va a ser 0 en todo momento, donde en dicha fase tenemos constantemente los estados de S1 en verde y S2 en rojo, formando el estado inicial del sistema.

- 2) Si el carSensor se activa y estamos en la Fase 0, se va a realizar todo el ciclo (diseñado en la foto de la primera página del documento).

- Esto lo implementamos a través de “if carSensor and (preFase = 0) then 1 else if 0 < preFase and preFase < 8 then preFase + 1”. A través de la línea 2 de la “Imagen 2” hacemos que si se activa el carSensor y la Fase = 0, se pasa a la Fase siguiente (Fase = 1). La línea siguiente nos permite avanzar en el ciclo, donde se dice que mientras la Fase sea mayor que 0 y menor que 8 se va sumando 1 a la Fase en cada tick. Cuando estemos en el tick 8, se vuelve al estado inicial, permitiendo completar el ciclo.

- 3) Si el carSensor se activa pero no estamos en la Fase 0, el ciclo continúa sin volver a reiniciarse y una vez vuelto al estado inicial el sistema sí reacciona cuando se activa el carSensor:

- Esto es posible gracias a la parte de “preFase = 0” de la línea 2 de la “Imagen 2”, ya que para que se haga caso al sensor únicamente se puede estar en la Fase 0 del ciclo, permitiendo así que este nunca se corte si se pulsa el botón y dicho ciclo está transcurriendo.

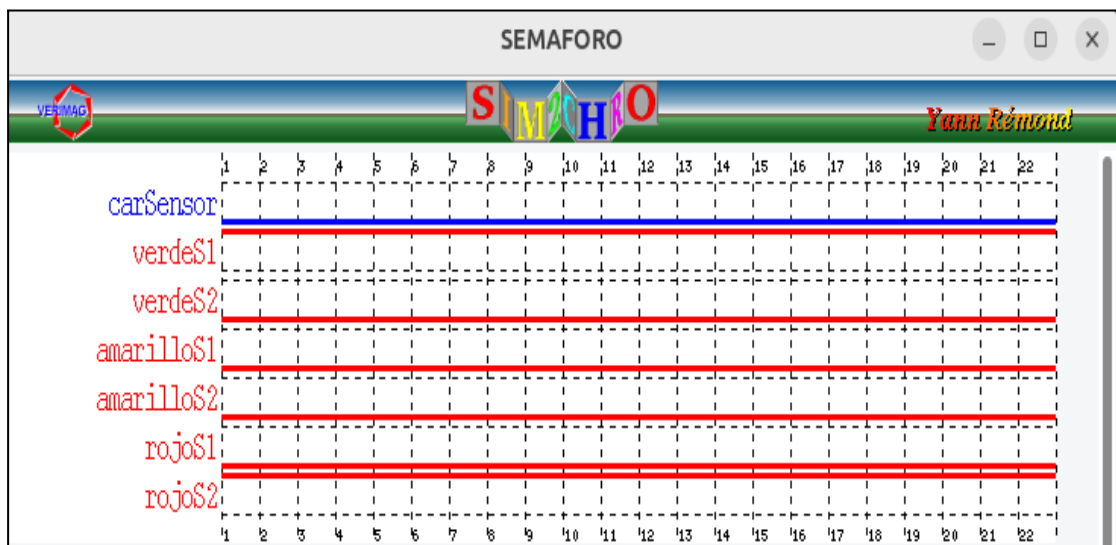
Además de analizar diferentes escenarios que íbamos a tener, hicimos un seguimiento de los colores (estados) en los que iban a estar los semáforos en cada tick. Lo pudimos hacer gracias al pensamiento inicial de la “Imagen 1” que tuvimos para plasmar la idea básica del problema. Con esto asignamos los valores a las variables verdeS1, verdeS2, amarilloS1, amarilloS2, rojoS1 y rojoS2 en las diferentes fases.

RESULTADOS:

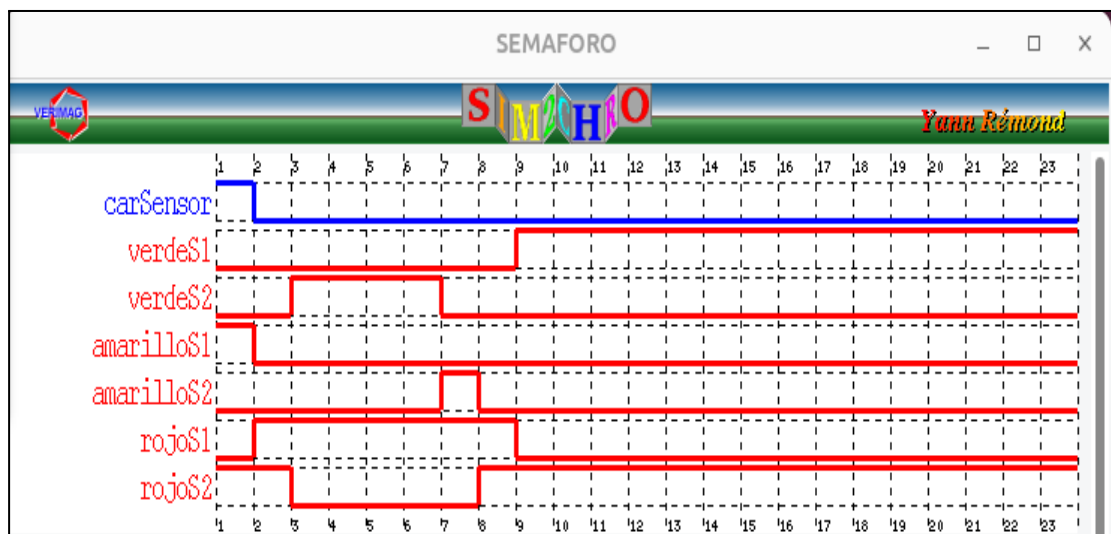
El código se puede encontrar en *semaforo.lus* y para visualizarlo basta con poner en la terminal “*luciole semaforo.lus SEMAFORO*”

Para comprobar que el código funciona correctamente vamos a analizar las salidas de cada escenario en sim2chro, recordemos que los resultados deben coincidir con los escenarios que hemos explicado cuando los explicamos en las anteriores páginas:

- 1) **Escenario 1 (estado inicial):** “Si no se detecta ningún coche el semáforo S1 permanecerá en verde y el semáforo S2 en rojo”:

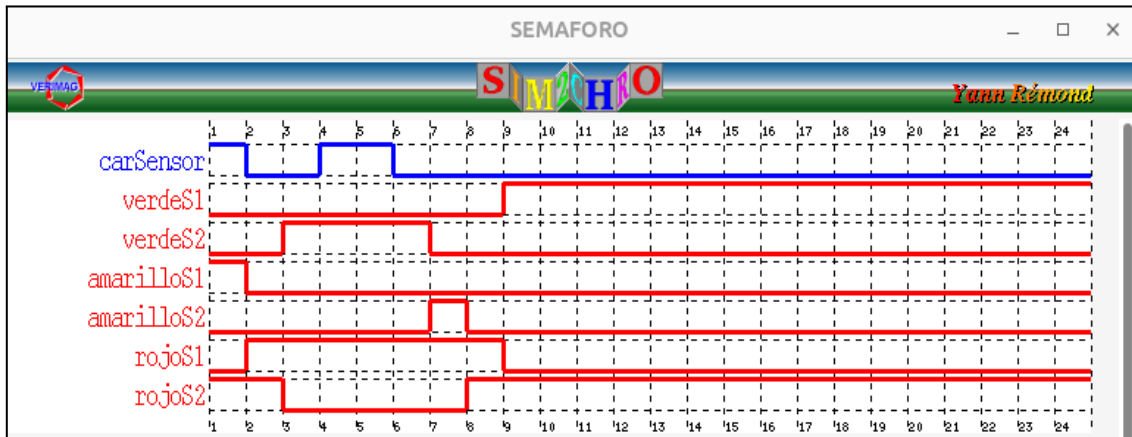


- 2) **Escenario 2:** “Si se detecta un coche se realiza todo el ciclo donde el semáforo S1 se convierte a rojo y el semáforo S2 se convierte en verde. Una vez acabado el ciclo se vuelve al estado inicial”:



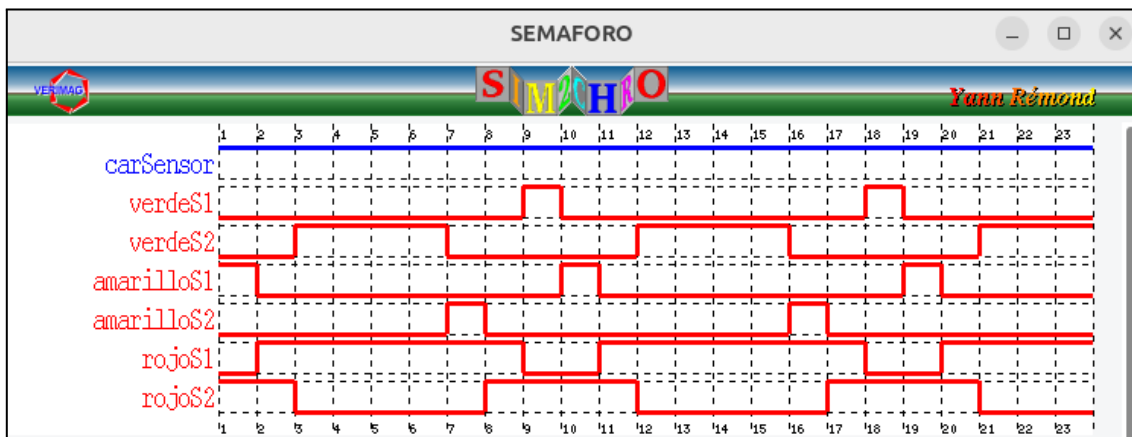
-> Podemos apreciar que cuando se acaba el ciclo se vuelve al estado inicial, con posibilidad de iniciar otro ciclo si se detecta otro coche o seguir en el estado inicial constantemente

- 3) **Escenario 3:** “Se detecta un coche y se inicia el ciclo, pero mientras transcurre dicho ciclo se vuelve a detectar otro coche”:



-> Se puede ver que aunque se detecte otro coche en el medio del transcurso de un ciclo dicho ciclo se ejecuta hasta terminar y una vez finalizado es cuando el sistema vuelve a reaccionar ante la detección de otro coche.

- 4) **Escenario 4:** “Se detectan coches indefinidamente”:



-> Transcurre un ciclo tras otro hasta que se dejen de detectar coches, donde ahí se vuelve al estado inicial.

ACTIVIDAD 2: Intentar aplicar este problema al caso de tres semáforos.

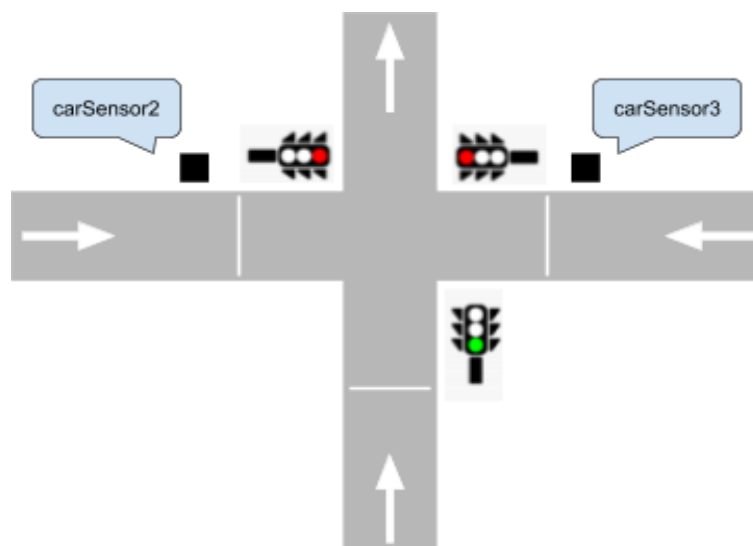
Para resolver este problema hemos decidido que el cruce sea esta vez de 4 calles, en donde todas son de sentido único pero los 3 carriles tienen que ir hacia la calle superior, por lo que es necesario que 3 semáforos regulen el paso.

- 1) Para ello, por defecto el semáforo de abajo (al que llamamos S1) estará por defecto en verde y los otros dos en rojo. Es el que regula el carril que va desde abajo.
- 2) El semáforo de la izquierda es el semáforo S2 y tendrá un sensor sin prioridad (carSensor2), detectando cuando pasa un coche para que este se pueda poner en verde y los dos otros semáforos estén en rojo, este semáforo regula el carril que va desde la izquierda.
- 3) Por último el semáforo de la derecha S3, también presenta un sensor (carSensor3) pero este tiene prioridad al sensor carSensor2. Cuando se detecte que un coche llega al sensor, este semáforo se pondrá en verde y los otros dos deben estar en rojo. Este semáforo regula el carril que viene desde la derecha.

Para que no haya conflictos entre los dos sensores, en el caso de que se detecten coches en los dos sensores a la vez, primero se pondrá en verde el semáforo S3 que tiene el sensor carSensor3 con prioridad y cuando termine, ya se puede poner en verde el semáforo S2 con carSensor2 (por eso decimos que este sensor no tiene prioridad).

Hay que destacar también que hemos decidido que estos semáforos se comporten igual que en la Actividad 1, es decir, para pasar de verde a rojo hay un estado ambar en el medio; antes de que un semáforo se ponga en verde, debe haber una etapa de transición en donde los 3 semáforos estén en rojo por si justo había un coche cruzando y por último que los semáforos duren 4 segundos en verde.

En la siguiente imagen se puede ver la realización de un dibujo del problema hecho por nosotros para que se vea más claro:



SOLUCIÓN DEL PROBLEMA:

Lo primero que hicimos para resolver este problema fue dividirlo en los posibles casos que podrían ocurrir, esto son las 4 combinaciones posibles que suceden al activarse/desactivarse los sensores carSensor2 y carSensor3:

	CASO inicial	CASO 2	CASO 3	CASO 4
carSensor2	0	1	0	1
carSensor3	0	0	1	1

- 1) **Caso inicial:** Ningún sensor se activó y el semáforo S1 está siempre en verde, mientras que S2 y S3 están siempre en rojo al no haber coches.
- 2) **Caso 2:** Se ha activado el sensor carSensor2 pero no el carSensor3, por lo que el semáforo S2 se debe encender 4 segundos mientras S1 y S3 están en rojo, después S1 se encenderá y S2 y S3 estarán en rojo.
- 3) **Caso 3:** Se ha activado el sensor carSensor3 pero no el carSensor2, por lo que el semáforo S3 se debe encender 4 segundos mientras S1 y S2 están en rojo, después S1 se encenderá y S2 y S3 estarán en rojo.
- 4) **Caso 4:** Este es el más complejo, los dos sensores se activan, por lo que primero el semáforo S3 se queda 4 segundos en verde (ya que carSensor3 tiene prioridad), mientras S1 y S2 están en rojo. Después le toca a S2 ponerse en verde 4 segundos mientras S1 y S3 están en rojo, por último S1 se pone en verde mientras S2 y S3 están en rojo.

Dividir el problema en casos ha sido muy importante ya que nos facilitó mucho trabajo, porque con esto nos hemos dado cuenta, que el código para el Caso 2 es prácticamente idéntico a la actividad 1 con los semáforos S1 y S2 ya que S3 está siempre en rojo. Y para el Caso 3 sería lo mismo solo que solamente utilizando los semáforos S3 y S1, estando siempre S2 en rojo. Por lo que solo nos faltaría por implementar el código del caso 4, que en vez de tener 8 fases, tendrá 14.

Así que aplicamos esta lógica para hacer el ciclo de las fases en cada caso:

```
preFase = 0 -> pre(Fase);
preCaso = 1 -> pre(caso);

caso = if (preFase = 0) then
  if (carSensor2 and carSensor3) then 4
  else if carSensor3 then 3
  else if carSensor2 then 2
  else 1
  else preCaso;

Fase = if (preFase = 0 and caso > 1) then 1
  else if (caso = 2 or caso = 3) and (preFase < 8) then preFase + 1
  else if (caso = 4) and (preFase < 14) then preFase + 1
  else 0;
```

Ahora solo nos queda implementar la lógica de cuando hacemos que se active cada variable de salida, como hay 3 semáforos, las variables de salida van a ser: *verdeS1*, *verdeS2*, *verdeS3*, *amarilloS1*, *amarilloS2*, *amarilloS3*, *rojoS1*, *rojoS2* y *rojoS3*. Pero antes de escribir el código de las diferentes variables de salida, hicimos un esquema de que semáforos se tendrían que encender en cada caso, para que solo tuviésemos que seguir estos pasos al escribir el código:

*(CS2= carSensor2, CS3=carSensor3)

Caso 1 $\rightarrow S1(\text{verde}), S2(\text{rojo}), S3(\text{rojo})$

Caso 2 (CS2) $\left\{ \begin{array}{l} 0 \rightarrow S1(\text{verde}), S2(\text{rojo}), S3(\text{rojo}) \\ 1 \rightarrow S1(\text{amarillo}), S2(\text{rojo}) \\ 2 \rightarrow S1(\text{rojo}), S2(\text{rojo}) \\ 3 \rightarrow S1(\text{rojo}), S2(\text{verde}) \\ 4 \rightarrow \text{''} \text{''} \\ 5 \rightarrow \text{''} \text{''} \\ 6 \rightarrow \text{''} \text{''} \\ 7 \rightarrow S1(\text{rojo}), S2(\text{amarillo}) \\ 8 \rightarrow S1(\text{rojo}), S2(\text{rojo}) \end{array} \right.$

• S3 siempre en rojo

Caso 3 (CS3) $\left\{ \begin{array}{l} 0 \rightarrow S1(\text{verde}), S2(\text{rojo}), S3(\text{rojo}) \\ 1 \rightarrow S1(\text{amarillo}), S3(\text{rojo}) \\ 2 \rightarrow S1(\text{rojo}), S3(\text{rojo}) \\ 3 \rightarrow S1(\text{rojo}), S3(\text{verde}) \\ 4 \rightarrow \text{''} \text{''} \\ 5 \rightarrow \text{''} \text{''} \\ 6 \rightarrow \text{''} \text{''} \\ 7 \rightarrow S1(\text{rojo}), S3(\text{amarillo}) \\ 8 \rightarrow S1(\text{rojo}), S3(\text{rojo}) \end{array} \right.$

• S2 siempre en rojo

Caso	
4	
(CS3	0 → S1 (verde), S2 (rojo), S3 (rojo)
y	
(CS2)	1 → S1 (amarillo), S3 (rojo)
	2 → S1 (rojo), S3 (rojo)
	3 → S1 (rojo), S3 (verde)
	4 → " "
	5 → " "
	6 → " "
	7 → S1 (rojo), S3 (amarillo)
	8 → S1 (rojo), S2 (rojo), S3 (rojo)
	9 → S1 (rojo), S2 (verde), S3 (rojo)
	10 → " "
	11 → " "
	12 → " "
	13 → S1 (rojo), S2 (amarillo), S3 (rojo)
	14 → S1 (rojo), S2 (rojo), S3 (rojo)

Una vez ya sabemos los valores de las posibles salidas de cada semáforo (verdeS1, verdeS2, verdeS3, amarilloS1, amarilloS2, amarilloS3, rojoS1, rojoS2, rojoS3) implementarlos al código es más sencillo, solo hay que poner en cada variable de salida en qué casos y en qué fases se activa:

```

verdeS1 = (Fase = 0);
amarilloS1 = (Fase = 1);
rojoS1 = (Fase > 1);

verdeS2 = (caso = 2 and Fase >= 3 and Fase <= 6) or
(caso = 4 and Fase >= 9 and Fase <= 12);
amarilloS2 = (caso = 2 and Fase = 7) or
(caso = 4 and Fase = 13);
rojoS2 = not (verdeS2 or amarilloS2);

verdeS3 = (caso = 3 and Fase >= 3 and Fase <= 6) or
(caso = 4 and Fase >= 3 and Fase <= 6);
amarilloS3 = (caso = 3 and Fase = 7) or
(caso = 4 and Fase = 7);
rojoS3 = not (verdeS3 or amarilloS3);

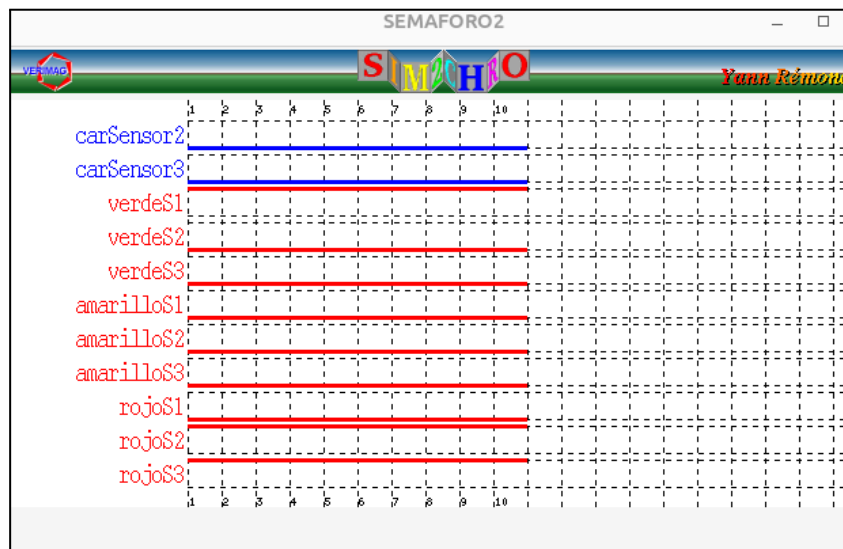
```

RESULTADOS:

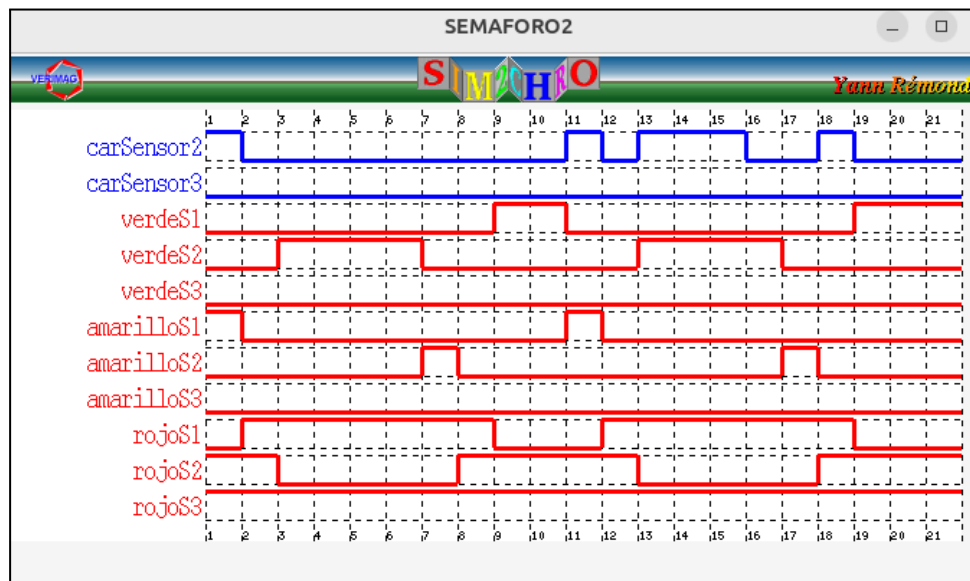
El código se puede encontrar en *semaforo2.lus* y para visualizarlo basta con poner en la terminal “luciole semaforo2.lus SEMAFORO2”

Para comprobar que el código funciona correctamente vamos a analizar las salidas de cada caso en sim2chro, recordemos que los resultados deben coincidir con los casos que hemos explicado cuando hicimos la tabla de cada combinación:

- 1) **Caso inicial:** “Ningún sensor se activó y el semáforo S1 está siempre en verde, mientras que S2 y S3 están siempre en rojo al no haber coches.”

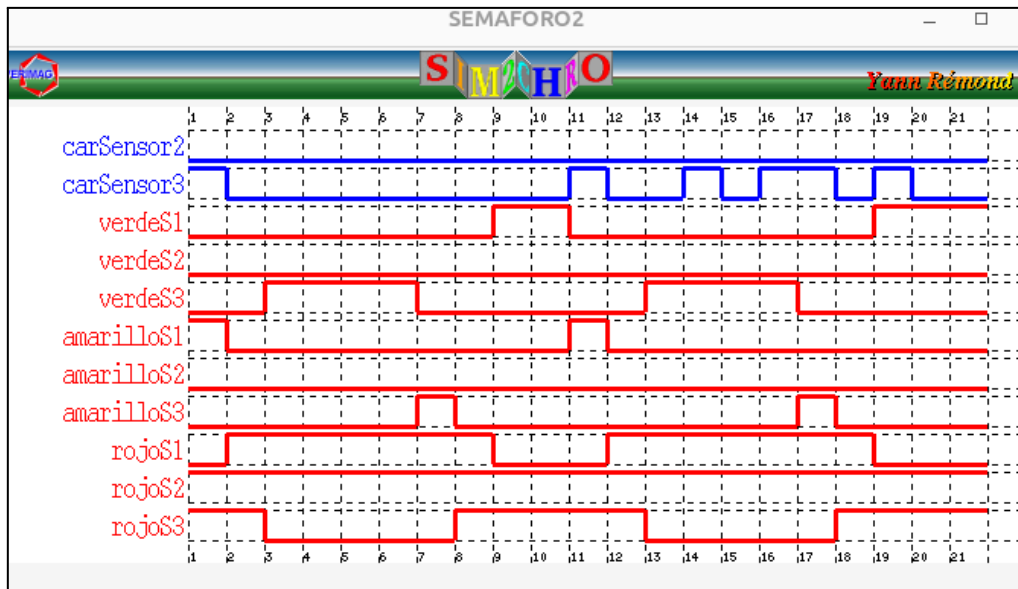


- 2) **Caso 2:** “Se ha activado el sensor carSensor2 pero no el carSensor3, por lo que el semáforo S2 se debe encender 4 segundos mientras S1 y S3 están en rojo, después S1 se encenderá y S2 y S3 estarán en rojo.”



-> Se puede observar como aunque se active varias veces carSensor2 no se rompe el ciclo del semáforo S2 y también funciona en el tick inicial.

- 3) **Caso 3:** “Se ha activado el sensor carSensor3 pero no el carSensor2, por lo que el semáforo S3 se debe encender 4 segundos mientras S1 y S2 están en rojo, después S1 se encenderá y S2 y S3 estarán en rojo.”



-> Se puede observar como aunque se active varias veces carSensor3 no se rompe el ciclo del semáforo S3 y también funciona en el tick inicial.

- 4) **Caso 4:** “Este es el más complejo, los dos sensores se activan, por lo que primero el semáforo S3 se queda 4 segundos en verde (ya que carSensor3 tiene prioridad), mientras S1 y S2 están en rojo. Después le toca a S2 ponerse en verde 4 segundos mientras S1 y S3 están en rojo, por último S1 se pone en verde mientras S2 y S3 están en rojo.”

IMAGEN1: Los dos sensores se activan a la vez

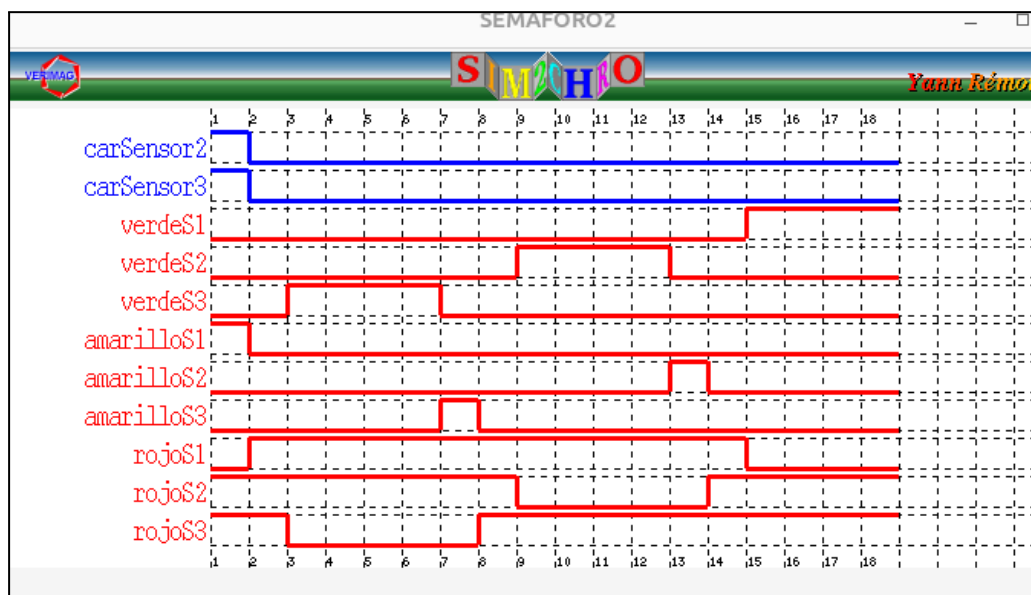


IMAGEN2: Los dos sensores se activan a la vez, pero en medio del ciclo se detectan coches en los sensores pero el ciclo no varía.

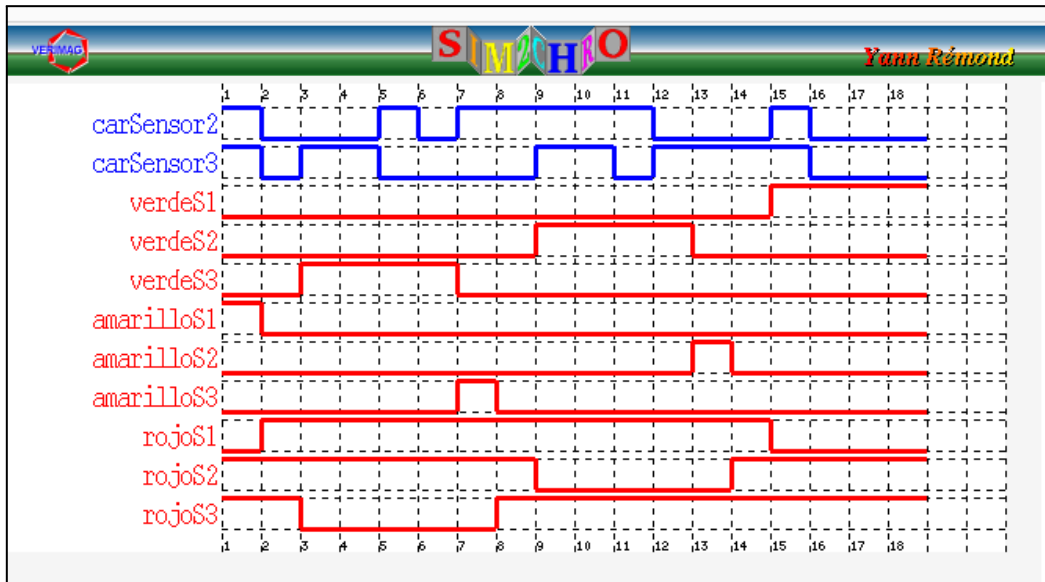


IMAGEN3: Se activa solo S2, pero en medio del ciclo el sensor con prioridad carSensor3 se dispara, pero al haber empezado ya el ciclo no se corta, funcionando correctamente.

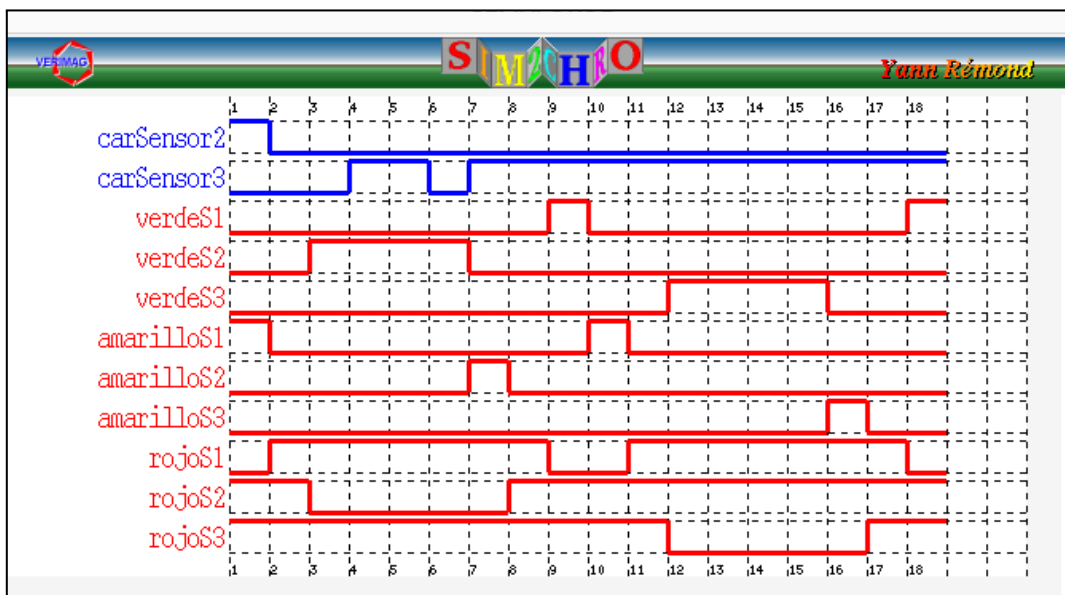


IMAGEN4: Se activa solo S3, pero en medio del ciclo el sensor con prioridad carSensor2 se dispara, pero al haber empezado ya el ciclo no se corta, funcionando correctamente.

